



DSA MARATHON

Week 1 — Foundations of Programming

Textbook-Style Notes for C++ and Python

13 Chapters • Definitions • Syntax • Worked Examples
Key Points • Common Mistakes • C++ vs Python Comparisons
With a dedicated Rough Work column on every page

Prepared for DSA Marathon Batch — Week 1 Complete Revision Guide
NextGen Data Minds

Table of Contents

1. Input and Output
2. Printing Numbers and Text
3. Variables
4. Variable Naming Conventions
5. Taking Input and Output — Applied Patterns
6. Types of Operators
7. All Data Structures — A Foundational Overview
8. ASCII Values
9. Hierarchy of Operators (Operator Precedence)
10. Arithmetic Operators
11. Increment and Decrement Operators
12. Operators, the Modulus Operator, and Comments
13. Updating and Assignment Operators

CHAPTER 1

ROUGH WORK

Input and Output

Every useful program is built around a simple loop: it receives data, processes it, and produces a result. The receiving half is called input and the producing half is called output. This chapter builds the foundation you will use in literally every program you write for the rest of this marathon.

DEFINITION

Input is any data a program receives from an external source (keyboard, file, network, another program) during or before execution. Output is any data a program sends out — most commonly displayed on the screen (the console or terminal), but it can also be written to a file or sent over a network.

1.1 The I/O Toolkit in Each Language

C++ handles console I/O through streams, defined in the header `<iostream>`. A stream is an abstraction for a flow of data. `cout` ("character output") is the standard output stream, and `cin` ("character input") is the standard input stream. Both live inside the `std` namespace, which is why you either write `std::cout` or add `using namespace std;` at the top of the file.

Python keeps things simpler at the surface: `print()` is a built-in function that writes to standard output, and `input()` is a built-in function that reads one line of text from standard input. Neither requires an import, because they are part of Python's built-in namespace.

1.2 Syntax

C++

```
#include <iostream>
using namespace std;
int main() {
    int age;
    cout << "Enter your age: "; // OUTPUT: prompt the
    user
    cin >> age;                // INPUT: read a value
    into 'age'
    cout << "You are " << age << " years old" << endl;
    return 0;
}
```

Output: Enter your age: 20 You are 20 years old

PYTHON

```
age = int(input("Enter your age: ")) # INPUT (and
convert text -> int)
print("You are", age, "years old")    # OUTPUT
```

Output: Enter your age: 20 You are 20 years old

1.3 Why C++ needs main() and Python does not

C++ is a compiled language: the compiler translates your entire file into machine code before anything runs, and it needs one fixed, unambiguous entry point to start execution — that is exactly what `int main() { ... }` provides. Python is an interpreted language: the interpreter reads your file top to bottom and executes statements as it encounters them, so there is no special function required to 'start' the program — the top level of the file is the starting point.

Key Points

- C++: output uses `cout << value;`, input uses `cin >> variable;`. Both need `<iostream>`.
- Python: output uses `print(value)`, input uses `input()`, which always returns a string.
- In C++ you must declare a variable's type before `cin` can read into it. In Python, `input()` gives raw text that you convert yourself using `int()`, `float()`, etc.
- `endl` in C++ and the default behaviour of `print()` both move the cursor to a new line after output.

Common Mistakes & Gotchas

- Forgetting to convert the result of Python's `input()` before doing arithmetic — `input() + 1` raises a `TypeError` because you cannot add an `int` to a string.
- In C++, forgetting `#include <iostream>` or the `std::` prefix (when not using using namespace `std`;) causes a compile error.
- Using `cin >>` to read a full sentence — it stops at the first whitespace, so 'John Smith' would only capture 'John'. (Solved in Chapter 5.)

C++ vs Python

Aspect	C++	Python
Output statement	<code>cout << x;</code>	<code>print(x)</code>
Input statement	<code>cin >> x;</code>	<code>x = input()</code>
Required header/import	<code><iostream></code>	None (built-in)
Return type of input	Matches variable's declared type	Always <code>str</code>

Aspect	C++	Python
New line after output?	Only if endl or \n used	Automatic (unless end= is changed)

CHAPTER 2

ROUGH WORK

Printing Numbers and Text

Printing is how a program communicates with the person running it. Getting comfortable with formatting output — mixing text and numbers, controlling line breaks, and separating values — is a small skill that pays off constantly, especially when debugging DSA code by printing intermediate values.

2.1 Printing Plain Text and Numbers

C++ uses the insertion operator `<<` to push values, one after another, into the output stream. You can chain as many values as you like on a single `cout` statement. Python's `print()` function accepts any number of comma-separated arguments and automatically places a single space between them.

C++

```
cout << "Hello World" << endl;    // text
cout << 100 << endl;              // number
cout << "Score: " << 95 << endl;  // mixed, no automatic
                                spacing
```

Output: Hello World 100 Score: 95**PYTHON**

```
print("Hello World")    # text
print(100)              # number
print("Score:", 95)     # mixed, space auto-inserted
                        between args
```

Output: Hello World 100 Score: 95

2.2 Controlling Line Breaks and Separators

By default, `cout` never inserts a newline for you — you must add `endl` or the escape sequence `\n` explicitly. Python's `print()` does the opposite: it always adds a trailing newline unless you override it using the `end` keyword argument. Similarly, the space Python inserts between arguments can be changed using the `sep` keyword argument.

C++

```
cout << "A" << "B" << "C"; // no separators, no newline
```

```
// Output: ABC
```

PYTHON

```
print("A", end="")
print("B", end="")
print("C") # Output: ABC
print("A", "B", "C", sep="-") # Output: A-B-C
```

2.3 Formatted / Mixed Output

Neatly mixing variables into a sentence is common enough that both languages offer a dedicated tool. In C++, you typically just keep chaining <<. In Python, the modern and recommended tool is the f-string (formatted string literal), written as f"...{expression}...", which evaluates the expression inside the braces and inserts it directly into the string.

C++

```
int score = 95;
string name = "Aditi";
cout << name << " scored " << score << " marks" <<
endl;
```

Output: Aditi scored 95 marks

PYTHON

```
score = 95
name = "Aditi"
print(f"{name} scored {score} marks") # f-string:
clean and readable
```

Output: Aditi scored 95 marks

Key Points

- C++: chain values with <<; add endl or \n yourself for new lines.
- Python: print() auto-adds spaces between arguments and a newline at the end — both are controllable via sep and end.
- f-strings (f"text {var}") are the cleanest way to mix text and variables in Python; they also support inline expressions like f"{a+b}".
- endl flushes the output buffer, which is slightly slower than "\n" in performance-critical competitive programming code.

Common Mistakes & Gotchas

- Assuming Python's print() behaves like C++'s cout and does not auto-add spaces — it does, by default.
- Forgetting quotes around text in either language, e.g. writing cout << Hello; instead of cout << "Hello";, which causes a compile error.
- Using excessive endl in tight competitive-programming loops — prefer

"\n" for speed since endl forces a buffer flush.

C++ vs Python

Aspect	C++	Python
Chaining values	cout<<a<<b;	print(a, b)
Default separator	None	Single space
Default line ending	None	Newline
Change separator	Insert manually	sep="..."
Change line ending	Add/remove endl	end="..."
Embed variables in text	Chain with <<	f-strings: f"{var}"

CHAPTER 3

Variables

Variables are the most fundamental building block of any program — without a way to store and refer to data, no computation would be possible. This chapter explains what a variable really is at a conceptual level, and how each language's type system shapes the way you use them.

DEFINITION

A variable is a named reference to a piece of data stored in the computer's memory. The name lets your program read or change that data without knowing or caring about the underlying memory address.

3.1 Static Typing (C++) vs Dynamic Typing (Python)

C++ is a statically typed language: every variable is bound to a specific type (such as `int`, `float`, `char`, `bool`, `string`) at the moment it is declared, and this type cannot change for the lifetime of that variable. The compiler uses this information to allocate the exact right amount of memory and to catch type-related errors before the program ever runs.

Python is a dynamically typed language: a variable is really just a label (a name) that points to an object somewhere in memory, and that object carries its own type information. The same name can be re-pointed to a completely different type of object later in the program — this is legal, though generally discouraged for readability.

C++

```
int marks = 90;
float price = 49.99;
char grade = 'A';
string name = "Rahul";
```

ROUGH WORK

```
bool isPassed = true;
```

PYTHON

```
marks = 90          # int
price = 49.99       # float
grade = 'A'         # str (Python has no separate char
                    # type)
name = "Rahul"      # str
is_passed = True    # bool
```

3.2 Declaration vs Initialization

Declaration is the act of telling the compiler/interpreter that a variable exists (and, in C++, what type it is). Initialization is giving it its first value. In C++ these can happen separately — `int x;` declares `x` without a value, leaving it holding an unpredictable 'garbage' value until you assign one. In Python, a variable is created and initialized in the very same statement — there is no way to 'declare only' in Python, because a name simply does not exist until it is assigned something.

C++

```
int x;          // declared, but value is undefined
                ("garbage")
x = 5;          // now initialized
int y = 10;     // declared and initialized together
```

PYTHON

```
# there is no equivalent of "int x;" with no value in
Python
x = 5    # a variable is born the instant it's assigned
```

3.3 What a Variable Actually Stores

In C++, a variable of a primitive type (like `int`) directly holds its value in the memory allocated for it. In Python, every variable is a reference to an object; even a simple integer like `5` is stored as a full Python object with its own type and identity. This is why Python variables can be reassigned to any type freely — you are only ever changing what the name points to, not resizing a fixed memory slot.

Key Points

- A variable = a name bound to a value/object stored in memory.
- C++: statically typed, type fixed at declaration, must specify type explicitly.
- Python: dynamically typed, type inferred from the assigned value, can change later.
- C++ allows declaration without initialization; Python does not —

assignment creates the variable.

- Multiple variables can be declared/assigned together: `int a=1,b=2;` in C++, `a, b = 1, 2` in Python.

Common Mistakes & Gotchas

- Using an uninitialized C++ variable in a calculation — it compiles, but produces unpredictable results.
- Assuming Python variables have a fixed type just because they hold one type of value initially — re-assigning `x = "hello"` after `x = 5` is perfectly legal.
- Forgetting the semicolon at the end of a C++ statement — a very common source of compile errors for beginners.
- Mixing up C++ variable types accidentally, e.g. assigning a decimal value like 4.8 to an int, which silently truncates to 4 instead of rounding.

C++ vs Python

Aspect	C++	Python
Typing model	Static (fixed at declaration)	Dynamic (inferred, can change)
Type keyword required?	Yes (int, float, ...)	No
Declare without value	Allowed (int x;)	Not possible
Reassign to different type	Not allowed	Allowed
Memory model	Value stored directly (primitives)	Name references an object

CHAPTER 4

ROUGH WORK

Variable Naming Conventions

Choosing good variable names is not cosmetic — it directly affects how quickly you (and anyone reviewing your code) can understand what a program does. This chapter covers the hard rules every name must obey, and the soft conventions that professional codebases follow.

4.1 Hard Rules (Both Languages)

These rules are enforced by the compiler/interpreter — breaking them causes an error, not just a style complaint.

Key Points

- A name may contain letters, digits, and underscores only.
- A name must not start with a digit (e.g. 2value is illegal; value2 is fine).
- Names are case-sensitive: age, Age, and AGE are three distinct variables.
- A name cannot be a reserved keyword of the language (e.g. int, class, for, if, return).
- Spaces, hyphens, and most special characters (@ # \$ % - space) are not allowed inside a name.

4.2 Conventional Styles

Beyond the hard rules, each language community has developed its own convention — a style that isn't enforced by the compiler but is expected by other programmers reading your code.

C++ traditionally favors camelCase for variables and function names (first

word lowercase, each subsequent word capitalized, no separators) — e.g. studentMarks, totalPrice. Class names typically use PascalCase (every word capitalized) — e.g. StudentRecord.

Python's official style guide, PEP 8, recommends snake_case for variables and functions (all lowercase, words separated by underscores) — e.g. student_marks, total_price. Class names in Python still use PascalCase, matching C++ convention there.

C++

```
int studentMarks = 85;          // camelCase -
                                // conventional in C++
class StudentRecord { ... };    // PascalCase for a
                                // class/type name
```

PYTHON

```
student_marks = 85    # snake_case - PEP 8 standard
class StudentRecord: # PascalCase for a class, same as
    C++
    pass
```

4.3 Choosing Meaningful Names

A name should describe what the value represents, not how it is used internally. totalMarks is far better than tm or x, because it documents intent without needing a comment. The only common exception in DSA/competitive programming is short-lived loop counters, where single letters like i, j, k are a widely accepted convention.

Common Mistakes & Gotchas

- Using reserved keywords as names, e.g. int class = 5; — this fails to compile/parse in both languages.
- Starting a name with a digit: 2ndValue is invalid.
- Using a hyphen instead of an underscore: total-marks is parsed as 'total minus marks', not a single name.
- Choosing non-descriptive single-letter names for important, long-lived variables — this hurts readability beyond a 5-line loop.
- Mixing naming styles inconsistently within one file (e.g. totalMarks next to total_price in the same C++ program).

C++ vs Python

Aspect	C++	Python
Variable convention	camelCase	snake_case (PEP 8)
Class convention	PascalCase	PascalCase
Constant	ALL_CAPS (often with const)	ALL_CAPS by convention

Aspect	C++	Python
convention		
Case sensitivity	Yes	Yes
Reserved keywords block naming?	Yes	Yes

CHAPTER 5

ROUGH WORK

Taking Input and Output — Applied Patterns

Chapter 1 introduced the basic tools. This chapter drills into the real, everyday patterns you'll use constantly while solving DSA problems: reading a single number, reading several numbers on one line, and reading a full line of text.

5.1 Reading a Single Number

C++ requires a correctly-typed variable declared beforehand; cin then parses the incoming text and stores it as that type. Python's input() always returns a string, so you must explicitly wrap it in int() or float() to use it as a number.

C++

```
int n;
cin >> n;
cout << "Square is: " << n * n << endl;
```

Output: (input: 5) → Square is: 25

PYTHON

```
n = int(input())
print("Square is:", n * n)
```

Output: (input: 5) → Square is: 25

5.2 Reading Multiple Values on One Line

C++'s cin >> naturally skips over whitespace (spaces, tabs, newlines) between tokens, so chaining cin >> a >> b; reads two space-separated values in one go. Python has no direct equivalent operator, so the idiomatic approach is: read the whole line with input(), split it into pieces with .split(), then convert every piece to the desired type using map().

C++

```
int a, b;
cin >> a >> b;
cout << a + b;
```

Output: (input: 3 4) → 7

PYTHON

```
a, b = map(int, input().split())
print(a + b)
```

Output: (input: 3 4) → 7

For reading an entire array/list of unknown length from one line of space-separated numbers, the pattern extends naturally:

C++

```
int n;
cin >> n;
vector<int> arr(n);
for (int i = 0; i < n; i++) cin >> arr[i];
```

PYTHON

```
n = int(input())
arr = list(map(int, input().split())) # reads n
numbers from one line
```

5.3 Reading a Full Line (With Spaces)

Because `cin >>` stops at the first whitespace character, it cannot capture a sentence like "John Smith" in one variable — it would only capture "John". C++ solves this with the `getline(cin, str)` function, which reads everything up to (and discarding) the next newline character. Python's `input()` already reads a whole line by default, spaces included, so no special function is needed.

C++

```
string fullName;
getline(cin, fullName);
cout << fullName;
```

Output: (input: John Smith) → John Smith

PYTHON

```
full_name = input()
print(full_name)
```

Output: (input: John Smith) → John Smith

Key Points

- C++: `cin >>` reads token by token, automatically skipping whitespace; use `getline` for full lines.
- Python: `input()` always reads one full line; use `.split()` to break it into tokens.
- The pattern `map(int, input().split())` is the standard Python idiom for reading several integers from one line — memorize it.
- Mixing `cin >>` and `getline` in the same C++ program can cause a subtle bug: a leftover newline character in the buffer gets immediately consumed by the next `getline`, producing an empty string.

Common Mistakes & Gotchas

- In C++, calling `getline(cin, str)` right after `cin >> someInt;` without clearing the leftover newline — the `getline` call appears to be 'skipped'.
- In Python, forgetting that `input().split()` splits on any amount of whitespace by default, which is usually what you want, but can surprise you with extra/irregular spacing.
- Forgetting to convert types after `map(int, ...)` is required — using raw `input().split()` gives a list of strings, not numbers.

C++ vs Python

Task	C++	Python
Read one integer	<code>cin >> n;</code>	<code>n = int(input())</code>
Read two integers, one line	<code>cin >> a >> b;</code>	<code>a, b = map(int, input().split())</code>
Read array of n integers	loop with <code>cin >> arr[i]</code>	<code>list(map(int, input().split()))</code>
Read full line with spaces	<code>getline(cin, s);</code>	<code>s = input()</code>

CHAPTER 6

Types of Operators

An operator is a symbol (or keyword) that tells the computer to perform a specific operation on one or more values, called operands. This chapter gives you the full map of operator categories before we zoom into individual ones (arithmetic, precedence, increment/decrement, assignment) in later chapters.

DEFINITION

An operator acts on operands to produce a result — for example, in $a + b$, $+$ is the operator and a, b are the operands.

6.1 The Categories

Key Points

- Arithmetic — perform mathematical calculations: $+$ $-$ $*$ $/$ $\%$ (and Python-only $**$ $//$).
- Relational / Comparison — compare two values and produce a boolean result: $==$ $!=$ $>$ $<$ $>=$ $<=$.
- Logical — combine boolean expressions: AND, OR, NOT.
- Assignment — store a value into a variable: $=$ and its compound forms $+=$ $-=$ $*=$ $/=$ $\%=$.
- Bitwise — operate on individual binary bits: $\&$ $|$ $\wedge$ $\sim$ $<<$ $>>$.
- Increment / Decrement — increase or decrease a value by 1: $++$ and $--$ (C++ only).
- Ternary / Conditional — a compact inline if-else that returns a value ($? :$ in C++, a conditional expression in Python).
- Membership & Identity (Python-only) — test whether a value exists in a container (in) or whether two names refer to the exact same object (is).

ROUGH WORK

6.2 Logical Operators — Symbols vs Keywords

This is one of the most visible surface differences between the two languages. C++ uses punctuation symbols borrowed from C: `&&` for AND, `||` for OR, and `!` for NOT. Python, valuing readability, spells these out as English keywords: `and`, `or`, and `not`.

C++

```
if (a > 0 && b > 0) cout << "Both positive";
if (a < 0 || b < 0) cout << "At least one negative";
if (!isPassed) cout << "Failed";
```

PYTHON

```
if a > 0 and b > 0:
    print("Both positive")
if a < 0 or b < 0:
    print("At least one negative")
if not is_passed:
    print("Failed")
```

6.3 The Ternary Operator vs the Conditional Expression

C++ provides a dedicated ternary (three-operand) operator, `condition ? valueIfTrue : valueIfFalse`, which evaluates to one of two values based on a condition. Python has no symbol-based ternary operator; instead it offers a conditional expression with the syntax `valueIfTrue if condition else valueIfFalse`, which achieves the exact same result read in a more sentence-like order.

C++

```
int maxVal = (a > b) ? a : b;
```

PYTHON

```
max_val = a if a > b else b
```

6.4 Membership and Identity (Python only)

Python's `in` operator checks whether a value exists inside a container (list, string, set, dict), something C++ has no single-symbol equivalent for — you would normally write a loop or call a library function like `find()`. Python's `is` operator checks object identity (are these two names pointing to the exact same object in memory?), which is different from `==` (are these two values equal?).

PYTHON

```
nums = [1, 2, 3]
print(2 in nums) # True
```

```
print(5 in nums) # False
```

Common Mistakes & Gotchas

- Using `&&/||` in Python — these are bitwise-only symbols there and will not behave as logical AND/OR the way you expect.
- Using `and/or/not` in C++ — these are not valid logical operators in standard C++ syntax.
- Confusing Python's `is` with `==` — `is` checks identity, not value equality, and can give surprising results for numbers or strings.

C++ vs Python

Operator meaning	C++	Python
Logical AND	<code>&&</code>	<code>and</code>
Logical OR	<code> </code>	<code>or</code>
Logical NOT	<code>!</code>	<code>not</code>
Ternary / conditional	<code>c ? a : b</code>	<code>a if c else b</code>
Membership test	No direct operator	<code>in</code>
Identity test	No direct operator	<code>is</code>

CHAPTER 7

All Data Structures — A Foundational Overview

A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently. This chapter is a map of the territory — a preview of every structure you will meet in this marathon — not a deep dive; each one gets its own dedicated chapter later.

7.1 Linear Data Structures

Linear structures arrange elements sequentially, one after another.

Key Points

- Array — a fixed-size, contiguous block of same-type elements accessed by index. C++: raw arrays or the fixed-size STL array.
- Dynamic Array — an array that can grow/shrink at runtime. C++: vector. Python: the built-in list (dynamic by default).
- Linked List — elements ('nodes') linked via pointers/references rather than contiguous memory. C++: list (doubly linked) or built manually. Python: no built-in linked list.
- Stack — Last-In-First-Out (LIFO) access. C++: stack. Python: a plain list used with `append()`/`pop()`.
- Queue — First-In-First-Out (FIFO) access. C++: queue. Python: `collections.deque`.
- Deque — double-ended queue, insertion/removal from both ends. C++:

ROUGH WORK

deque. Python: collections.deque.

7.2 Associative / Key-Based Data Structures

Key Points

- Set — an unordered collection of unique values. C++: set (ordered, tree-based) or unordered_set (hash-based). Python: the built-in set (hash-based).
- Map / Dictionary — stores key-value pairs for fast lookup by key. C++: map (ordered) or unordered_map (hash-based). Python: the built-in dict (hash-based, preserves insertion order since Python 3.7).
- Tuple (Python-only built-in) — an ordered, immutable sequence, often used for fixed-size groupings like coordinates (x, y).

7.3 Non-Linear Data Structures (Preview)

These will each get a full dedicated session later in the marathon: Trees (hierarchical, parent-child relationships — e.g. Binary Trees, BSTs), Graphs (nodes connected by edges, modelling networks), and Heaps (specialized trees for fast access to the minimum/maximum element, underlying priority queues).

7.4 The Structures You Will Use Constantly

For everyday DSA work, C++'s vector<int> and Python's list are your default array-like structure — both are ordered, indexable, and resizable.

C++

```
#include <vector>
vector<int> nums = {1, 2, 3};
nums.push_back(4);
cout << nums[0];
```

Output: 1

PYTHON

```
nums = [1, 2, 3]
nums.append(4)
print(nums[0])
```

Output: 1

For fast key-based lookup, C++'s unordered_map and Python's dict are the default choice.

C++

```
#include <unordered_map>
unordered_map<string,int> ages;
ages["Tom"] = 25;
```

```
cout << ages["Tom"];
```

Output: 25

PYTHON

```
ages = {}
ages["Tom"] = 25
print(ages["Tom"])
```

Output: 25

Common Mistakes & Gotchas

- Assuming Python has a built-in linked list — it does not; list in Python is a dynamic array, not a linked list.
- Using C++'s map when you don't need ordering — unordered_map is faster for pure lookup use-cases (average $O(1)$ vs $O(\log n)$).
- Forgetting that a C++ raw array has a fixed size decided at creation, unlike vector or Python's list.

C++ vs Python

Concept	C++	Python
Dynamic array	vector	list
Stack (LIFO)	stack	list (append/pop)
Queue (FIFO)	queue	collections.deque
Unique-value set	set / unordered_set	set
Key-value map	map / unordered_map	dict
Immutable ordered group	std::tuple / const array	tuple

CHAPTER 8

ASCII Values

Underneath every character you see on screen, a computer stores a plain integer. Understanding this mapping unlocks a huge category of clean, efficient solutions to string and character problems in DSA.

DEFINITION

ASCII (American Standard Code for Information Interchange) is a character-encoding standard that assigns a unique integer, from 0 to 127, to each letter, digit, punctuation mark, and control character. For example: 'A' = 65, 'a' = 97, '0' = 48, and space = 32.

ROUGH WORK

8.1 Key Ranges to Memorize

Key Points

- Uppercase letters 'A'-'Z': 65 to 90.
- Lowercase letters 'a'-'z': 97 to 122.
- Digits '0'-'9': 48 to 57.
- The gap between an uppercase and its matching lowercase letter is always exactly 32 (e.g. $97 - 65 = 32$) — the basis for fast case conversion tricks.
- Space character: 32. Newline ('\n'): 10.

8.2 Converting Between Character and Code

In C++, a char is, at the machine level, already a small integer — assigning it to an int variable automatically reveals its ASCII code, and assigning an integer to a char variable converts it back. Python keeps characters and integers as visibly separate types (there is no 1-character 'char' type — a single character is just a string of length 1), so it provides two explicit built-in functions: ord(ch) returns the ASCII/Unicode code of a character, and chr(code) returns the character for a given code.

C++

```
char c = 'A';
int code = c; // implicit conversion: code = 65
char back = 66; // implicit conversion: back = 'B'
cout << code << " " << back;
```

Output: 65 B

PYTHON

```
code = ord('A') # 65
back = chr(66) # 'B'
print(code, back)
```

Output: 65 B

8.3 Practical Applications in DSA

A very common pattern is mapping a lowercase letter to an index between 0 and 25 for frequency-counting problems (e.g. "is this an anagram?"). This works because subtracting the code of 'a' from any lowercase letter's code gives its 0-based position in the alphabet.

C++

```
int freq[26] = {0};
string s = "hello";
```

```
for (char c : s) freq[c - 'a']++; // 'h'-'a'=7,
'e'-'a'=4, etc.
```

PYTHON

```
freq = [0] * 26
s = "hello"
for c in s:
    freq[ord(c) - ord('a')] += 1 # 'h'-'a'=7,
'e'-'a'=4, etc.
```

The same 32-gap trick lets you convert case without a lookup table: adding 32 to an uppercase letter's code gives its lowercase form, and subtracting 32 from a lowercase letter's code gives its uppercase form.

C++

```
char upper = 'a' - 32; // 'A'
char lower = 'A' + 32; // 'a'
```

PYTHON

```
upper = chr(ord('a') - 32) # 'A'
lower = chr(ord('A') + 32) # 'a'
```

Common Mistakes & Gotchas

- Forgetting that ASCII only covers 0-127 — extended characters, emoji, and non-English scripts need Unicode, which Python's `ord()/chr()` handle transparently but C++'s plain `char` does not.
- Trying to use `ord()` on a multi-character Python string — it only accepts a string of length exactly 1 and raises a `TypeError` otherwise.
- Off-by-one errors when computing `c - 'a'` for uppercase letters — remember uppercase and lowercase have different base offsets (65 vs 97).

C++ vs Python

Task	C++	Python
Character → code	<code>int code = ch;</code>	<code>ord(ch)</code>
Code → character	<code>char ch = code;</code>	<code>chr(code)</code>
Has separate char type?	Yes (<code>char</code>)	No (<code>length-1 str</code>)
Default encoding	ASCII / raw byte value	Unicode (ASCII is a subset)

CHAPTER 9

ROUGH WORK

Hierarchy of Operators (Operator Precedence)

When an expression contains more than one operator, the computer needs a consistent rule to decide which operation happens first. That rule is called operator precedence, and it is one of the most common sources of subtle bugs for beginners — and, occasionally, for experienced programmers too.

DEFINITION

Operator precedence defines the order in which operators are evaluated in an expression with multiple operators. Operators with higher precedence are evaluated before operators with lower precedence. When two operators share the same precedence, associativity (left-to-right or right-to-left) decides the order between them.

9.1 Precedence Table (High to Low)

Precedence Levels

Level	Operators	Category
1 (highest)	()	Parentheses / grouping
2	++ -- ! not (unary)	Unary operators
3	* / % //	Multiplicative
4	+ -	Additive
5	< <= > >=	Relational
6	== !=	Equality
7	&& / and	Logical AND
8	/ or	Logical OR
9 (lowest)	= += -= *= /= %=	Assignment

This ordering is essentially identical between C++ and Python for the operators they share; Python simply adds a few extra operators (like **, which actually binds even tighter than unary minus) that C++ doesn't have as symbols.

9.2 Worked Example

C++

```
int result = 2 + 3 * 4; // multiplication first:
3*4=12, then 2+12
cout << result;          // 14, NOT (2+3)*4 = 20
bool ok = 5 > 3 && 2 < 4; // relational before logical
AND
cout << ok;              // true
```

Output: 14 1

PYTHON

```
result = 2 + 3 * 4    # 14, not 20
ok = 5 > 3 and 2 < 4 # True
```

Output: 14 True

9.3 Associativity

Most binary operators (like +, -, *, /) are left-associative: when operators of equal precedence appear together, they are evaluated left to right. Assignment, by contrast, is right-associative — this is what makes chained assignment like `a = b = c = 5`; work: it evaluates as `a = (b = (c = 5))`.

C++

```
int a = 10 - 3 - 2; // left-to-right: (10-3)-2 = 5, not
10-(3-2)=9
cout << a;
```

Output: 5

PYTHON

```
a = 10 - 3 - 2 # left-to-right: (10-3)-2 = 5
print(a)
```

Output: 5

Key Points

- Parentheses () always win — use them whenever you're unsure of default precedence, or simply to make intent obvious to a reader.
- Multiplication/division/modulus bind tighter than addition/subtraction.
- Relational operators (>, <) bind tighter than logical operators (&&, || / and, or).
- Assignment has the lowest precedence of the operators covered here, and is right-associative.
- Python's ** (exponentiation) is right-associative and binds very tightly: `2 ** 3 ** 2` evaluates as `2 ** (3 ** 2) = 512`.

Common Mistakes & Gotchas

- Assuming addition happens before multiplication because it's 'read first'

left to right — precedence, not reading order, decides this.

- Writing conditions like `a && b == c` and expecting `(a && b) == c` — actually `==` binds tighter, so it's `a && (b == c)`.
- Not realizing chained comparisons work differently: Python allows `1 < x < 10` directly as a natural range check, while C++ evaluates it left to right as `(1 < x) < 10`, comparing a boolean (0 or 1) to 10.

Arithmetic Operators

Arithmetic operators perform mathematical calculations and are the operators you'll reach for most often. This chapter covers the shared core set, plus the extra operators Python provides, and the single most important gotcha in this entire notes set: how division behaves differently across the two languages.

10.1 The Shared Core Set

Core Arithmetic Operators (a=7, b=2)

Operator	Meaning	Example
+	Addition	$a + b = 9$
-	Subtraction	$a - b = 5$
*	Multiplication	$a * b = 14$
/	Division	see 10.2 below — differs by language!
%	Modulus (remainder)	$a \% b = 1$

C++

```
int a = 7, b = 2;
cout << a + b << " " << a - b << " " << a * b <<
endl; // 9 5 14
cout << a / b << endl; // 3 (integer division)
cout << a % b << endl; // 1 (remainder)
```

Output: 9 5 14 3 1

PYTHON

```
a, b = 7, 2
print(a + b, a - b, a * b) # 9 5 14
print(a / b) # 3.5 (true division, always returns a
float)
print(a // b) # 3 (floor division)
print(a % b) # 1 (remainder)
print(a ** b) # 49 (exponent: a to the power b)
```

Output: 9 5 14 3.5 3 1 49

10.2 The Critical Difference: How '/' Behaves

This is arguably the single most important cross-language gotcha covered in these notes. In C++, the behaviour of / depends entirely on the types of the operands: if both operands are integers, C++ performs integer division and simply discards (truncates) any decimal part — so $7 / 2$ gives 3, not 3.5. If either operand is a floating-point type, C++ performs true decimal division.

Python 3 made a deliberate design choice here: the plain `/` operator always performs true division and returns a float, regardless of whether the operands are integers — so `7 / 2` gives 3.5. If you specifically want the old C-style truncating integer division, Python provides a separate operator for it: `//` (floor division), so `7 // 2` gives 3.

C++

```
cout << 7 / 2;    // 3 (int / int -> int, truncated)
cout << 7.0 / 2; // 3.5 (float involved -> true
division)
```

Output: 3 3.5

PYTHON

```
print(7 / 2)    # 3.5 (always true division in Python 3)
print(7 // 2)   # 3 (floor division -> matches C++'s
int/int)
```

Output: 3.5 3

10.3 Extra Operators Python Provides

Python offers two arithmetic operators with no direct single-symbol equivalent in C++: `**` for exponentiation (`a ** b` means `a` raised to the power `b`), and `//` for floor division (division that always rounds down towards negative infinity, discarding the remainder).

To raise a number to a power in C++, you instead call the library function `pow(a, b)` from `<cmath>`, which returns a double (a floating-point value) even if the inputs are integers.

C++

```
#include <cmath>
cout << pow(2, 10); // 1024 (returned as a double)
```

Output: 1024

PYTHON

```
print(2 ** 10) # 1024 (returned as an int, since inputs
were ints)
```

Output: 1024

Key Points

- C++ `/` truncates when both operands are integers; Python `/` always returns a float.
- Python's `//` (floor division) is the closest match to C++'s integer `/` — but floor division rounds toward negative infinity, which only differs from truncation when a negative number is involved.
- Python has native `**` for exponentiation; C++ needs `pow()` from

`<cmath>`.

- Modulus % works only on integer types in C++ (use `fmod()` for floating-point remainders); Python's % works on floats directly too.

Common Mistakes & Gotchas

- Porting a C++ formula that relies on integer division straight into Python and getting unexpectedly precise (float) results — always double check whether you need `/` or `//`.
- Forgetting `#include <cmath>` before calling `pow()` in C++.
- Assuming `pow(a, b)` in C++ returns an `int` — it returns a `double`, which can introduce floating-point rounding errors for large integer powers.
- Using % directly on floating-point numbers in C++ — this does not compile; you need `fmod()` from `<cmath>` instead.

CHAPTER 11

Increment and Decrement Operators

Incrementing and decrementing — adding or subtracting exactly 1 from a variable — happens so often (especially in loops and counters) that C++ gives it dedicated shorthand operators. Python deliberately omits them, which is a common early surprise for programmers coming from a C-family background.

DEFINITION

The increment operator (++) increases a variable's value by exactly 1. The decrement operator (--) decreases it by exactly 1. Both exist only in C++ among the two languages covered here.

11.1 Why Python Has No ++ or --

This is a deliberate design decision by Python's creators, not an oversight. Because ++ would be ambiguous with unary + applied twice, and because Python favors one obvious way to do things, incrementing in Python is always written explicitly as `x = x + 1` or, more idiomatically, using the compound assignment operator `x += 1` (covered fully in Chapter 13). Writing `x++` in Python is a syntax error.

C++

```
int x = 5;
x++; // post-increment, x becomes 6
++x; // pre-increment, x becomes 7
x--; // post-decrement, x becomes 6
cout << x;
```

Output: 6

PYTHON

```
x = 5
```

ROUGH WORK

```
x += 1 # x becomes 6 (equivalent effect to x++)
x += 1 # x becomes 7 (equivalent effect to ++x)
x -= 1 # x becomes 6 (equivalent effect to x--)
print(x)
# x++ or ++x would raise a SyntaxError in Python
```

Output: 6

11.2 Pre-Increment vs Post-Increment (C++ Only)

Both `++x` (pre-increment) and `x++` (post-increment) end up increasing `x` by exactly 1 — the final value of `x` is identical either way. The difference only becomes visible when the increment is used inline, as part of a larger expression, because they differ in what value they return:

Key Points

- Pre-increment (`++x`): increases `x` first, THEN returns the new (already-incremented) value.
- Post-increment (`x++`): returns the OLD value first, THEN increases `x` behind the scenes.

C++

```
int x = 5;
cout << x++ << endl; // prints 5 (the old value), THEN
x becomes 6
cout << x << endl;   // 6
cout << ++x << endl; // x becomes 7 FIRST, then 7 is
printed
```

Output: 5 6 7

11.3 Where This Matters Most: Loops

The most common home for `++/--` is the update step of a for loop. Python loops typically avoid manual counters altogether by iterating directly over a `range()`, which internally handles the counting for you — so the 'need' for `++` mostly disappears in idiomatic Python code.

C++

```
for (int i = 0; i < 5; i++) {
    cout << i << " ";
}
```

Output: 0 1 2 3 4

PYTHON

```
for i in range(5):
    print(i, end=" ")
```

Output: 0 1 2 3 4

Common Mistakes & Gotchas

- Trying to write `x++` or `x--` in Python — this is invalid syntax and will not run.
- Using post-increment inside a complex C++ expression without understanding it returns the OLD value — code like `arr[i++] = arr[i++]`; has undefined/confusing behaviour and should generally be avoided.
- Assuming `x += 1` and `x++` are 100% identical in C++ — for simple variables they have the same effect, but `++` only works on a single variable, while `+=` can add any amount, not just 1.

C++ vs Python

Aspect	C++	Python
Increment operator	<code>++x / x++</code>	Not available — use <code>x += 1</code>
Decrement operator	<code>--x / x--</code>	Not available — use <code>x -= 1</code>
Pre vs post distinction	Yes, differs in returned value	N/A
Typical loop counters	Manual, often with <code>i++</code>	Usually implicit via <code>range()</code>

CHAPTER 12

Operators, the Modulus Operator, and Comments

This chapter zooms into one specific arithmetic operator that deserves special attention — modulus — because of how frequently it appears in DSA problems and because of a genuinely tricky behavioural difference with negative numbers. It then covers comments, the tool you use to document your code for humans.

12.1 The Modulus Operator in Depth

DEFINITION

*The modulus operator (%) returns the remainder left over after dividing the left operand by the right operand. Formally, for integers a and b ($b \neq 0$), $a \% b$ satisfies $a = (a / b) * b + (a \% b)$, using the language's own definition of integer division $/$.*

Modulus is a workhorse operator in DSA, used constantly for: checking

ROUGH WORK

even/odd ($n \% 2 == 0$), extracting the last digit of a number ($n \% 10$), wrapping an index around the bounds of an array ($i \% \text{size}$, useful for circular buffers/hashing), and computing results modulo a large prime in competitive programming to prevent integer overflow.

C++

```
cout << (10 % 3); // 1
cout << (7 % 2); // 1 -> odd number
cout << (123 % 10); // 3 -> last digit of 123
```

Output: 1 1 3

PYTHON

```
print(10 % 3) # 1
print(7 % 2) # 1 -> odd number
print(123 % 10) # 3 -> last digit of 123
```

Output: 1 1 3

12.2 Modulus with Negative Numbers — A Genuine Gotcha

This is where C++ and Python truly disagree, and it is a favourite trick question in interviews. In C++ (since C++11), the sign of the result of `%` always matches the sign of the dividend (the left operand) — so $-7 \% 2$ gives -1 . In Python, the sign of the result always matches the sign of the divisor (the right operand) — so $-7 \% 2$ gives 1 , because Python's `%` is defined in terms of floor division, not truncating division.

C++

```
cout << (-7 % 2); // -1 (sign follows the dividend, -7)
cout << (7 % -2); // 1 (sign follows the dividend, 7)
```

Output: -1 1

PYTHON

```
print(-7 % 2) # 1 (sign follows the divisor, 2)
print(7 % -2) # -1 (sign follows the divisor, -2)
```

Output: 1 -1

Practical takeaway: if you are translating a modulus-based formula between C++ and Python (or vice versa) and negative numbers are possible, do not assume the result will match — verify it, or normalize it yourself, e.g. in Python: $((a \% b) + b) \% b$ guarantees a non-negative result regardless of the sign of a .

12.3 Comments

DEFINITION

A comment is text in source code that is completely ignored by the compiler/interpreter. Comments exist purely to communicate with human readers of the code — including your future self.

C++ supports two comment styles: `//` begins a single-line comment that extends to the end of the current line, and `/* ... */` marks a block comment that can span multiple lines. Python supports single-line comments with `#`; it has no true dedicated multi-line comment syntax, but triple-quoted strings (`'''...'''` or `"""..."""`) are conventionally used for the same purpose when the resulting string isn't assigned to anything (and are also the standard way to write docstrings).

C++

```
// this is a single-line comment
/* this is a
   multi-line block comment */
int x = 5; // an inline trailing comment
```

PYTHON

```
# this is a single-line comment
"""
This string, if not assigned to a variable,
is commonly used as a multi-line comment.
It is also the syntax used for docstrings.
"""
x = 5 # an inline trailing comment
```

Key Points

- `%` returns the remainder of integer division; extremely common in even/odd checks, digit extraction, and index wrapping.
- C++'s `%` result sign follows the dividend; Python's follows the divisor — verify this explicitly whenever negative operands are possible.
- C++ comments: `//` single-line, `/* */` block.
- Python comments: `#` single-line; triple-quoted strings are the conventional stand-in for block comments and are the real syntax for docstrings.
- Comments should explain why, not restate what the code already makes obvious.

Common Mistakes & Gotchas

- Assuming `-7 % 2` gives the same answer in both languages — it does not (`-1` in C++, `1` in Python).
- Trying to nest `/* */` block comments in C++ — this is not supported and causes a compile error at the first unmatched `*/`.

- Forgetting that a Python triple-quoted string used as a 'comment' is still technically a string literal — if placed where its value is used, it becomes a real docstring, not just a discarded comment.

CHAPTER 13

Updating and Assignment Operators

The final chapter of Week 1 covers assignment — the operator you will use more than any other — and its compact 'compound' forms, which combine an arithmetic operation with assignment into a single, readable step.

DEFINITION

The assignment operator (`=`) stores a value into a variable. A compound assignment operator combines an arithmetic (or bitwise) operation with assignment, updating a variable based on its own current value in a single expression — e.g. `x += 5` means "take `x`'s current value, add 5 to it, and store the result back into `x`," exactly equivalent to `x = x + 5`.

13.1 The Shared Core Set

C++

```
int x = 10;
x += 5; // x = x + 5 -> 15
x -= 3; // x = x - 3 -> 12
x *= 2; // x = x * 2 -> 24
x /= 4; // x = x / 4 -> 6 (integer division, since x is
int)
x %= 4; // x = x % 4 -> 2
cout << x;
```

Output: 2

PYTHON

```
x = 10
x += 5 # x = 15
x -= 3 # x = 12
x *= 2 # x = 24
x /= 4 # x = 6.0 (note: '/' always produces a float in
Python)
x %= 4 # x = 2.0
print(x)
```

ROUGH WORK

Output: 2.0

Notice the subtle consequence of Chapter 10's division rule showing up here again: in C++, $x / 4$ on an int performs integer division and keeps x as an int. In Python, $x / 4$ silently turns x into a float, even if it started as an int, because $/$ is built on top of $/$.

13.2 Extra Compound Operators in Python

Matching its extra arithmetic operators from Chapter 10, Python also provides $//$ (floor-divide and assign) and $**$ (exponentiate and assign). C++ has no single-symbol equivalent for either — you would write them out longhand, e.g. $x = \text{pow}(x, n)$;

PYTHON

```
x = 10
x //= 3 # x = 3 (floor divide and assign)
x **= 2 # x = 9 (raise to power and assign)
print(x)
```

Output: 9

13.3 Does $x += 5$ Always Equal $x = x + 5$?

For ordinary numeric variables (int, float), yes — the observable result is identical, and you can treat them as interchangeable. The distinction becomes meaningful later once you work with mutable container objects (like a Python list, or certain C++ STL containers): for those, $+=$ can modify the object in place (efficiently reusing the same memory), while $x = x + y$ may construct an entirely new object and rebind the name to it. This nuance will resurface once we start working with data structures in depth.

PYTHON

```
nums = [1, 2, 3]
nums += [4] # extends the SAME list object in place
print(nums) # [1, 2, 3, 4]
```

Output: [1, 2, 3, 4]

Key Points

- $x += n$ is shorthand for $x = x + n$; the same pattern applies for $-$, $*$, $/$, $\%$.
- Both languages share $+=$ $-$ $*$ $/$ $\%$; Python additionally offers $//$ and $**$.
- C++'s $/$ on integer variables stays an integer division; Python's $/$ always converts the result to a float.
- For simple numbers, $x += n$ and $x = x + n$ behave identically; for mutable objects, they can differ in whether the object is modified in place or

replaced.

Common Mistakes & Gotchas

- Expecting a Python int to stay an int after `/=` — it becomes a float, which can cause type errors later (e.g. when used as a list index).
- Using `//=` or `**=` in C++ — neither exists as an operator there; you must write the operation out longhand.
- Chaining compound assignment across multiple variables, e.g. attempting `a += b += 1;`, without understanding evaluation order — legal in C++ but can be genuinely confusing to read and is best avoided in favor of separate, explicit statements.

End of Week 1 Textbook Notes

Next up: Conditionals, Loops, and Functions.

NextGen Data Minds